OrthoPolyBasis — Complete API Reference

Version: 2.0 (HP-Z4 Architecture)
Repository: `OrthoPolyBasis-OK-Z4`

Table of Contents

---

1. Architecture Overview

Every polynomial family in this library follows a **4-Layer Architecture**:

```
+-------------------------------------------------------------------+
|  LAYER 4 — Integration / Quadrature (Orchestrator)                |
|  Imports from lower layers; provides Gauss-type quadrature,       |
|  function projection, and approximation.                           |
+-------------------------------------------------------------------+
|  LAYER 3 — Numerical (NumPy)                                      |
|  Fast double-precision evaluation via three-term recurrence.      |
|  Vectorized NumPy array support. Cached coefficients.             |
|  Dependency: numpy                                                 |
+-------------------------------------------------------------------+
|  LAYER 2 — High Precision (mpmath)                                |
|  Arbitrary-precision evaluation with configurable dps             |
|  (default 50 decimal places). Context-managed precision.          |
|  Dependency: mpmath                                                |
+-------------------------------------------------------------------+
|  LAYER 1 — Symbolic (SymPy)                                       |
|  Exact rational coefficients, symbolic expressions,               |
|  differentiation, and integration. Ground-truth for testing.      |
|  Dependency: sympy                                                 |
+-------------------------------------------------------------------+
```

Key Design Principle: Layers 1–3 are **independent** (no cross-imports). Only Layer 4 imports from lower layers as an orchestrator.

Installation

```
pip install sympy mpmath numpy
```

---

2. Chebyshev Polynomials T_n(x)

Domain: [-1, 1]
Weight function: w(x) = 1/sqrt(1 - x^2)
Recurrence: T_{n+1}(x) = 2x T_n(x) - T_{n-1}(x), with T_0 = 1, T_1 = x
Identity: T_n(cos(theta)) = cos(n * theta)

2.1 Layer 1 — Symbolic (SymPy)

Module: `chebyshev.symbolic`

Class: `ChebyshevSymbolic(n)` Constructs the exact Chebyshev polynomial T_n(x) using SymPy's `sp.chebyshevt()`.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree of polynomial

Properties:

Property	Return Type	Description
‘.expression‘	‘sympy.Basic‘	The symbolic expression for T_n(x)
‘.n‘	‘int‘	The degree of the polynomial

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> sympy.Basic‘	Substitute x into expression

```python
from chebyshev.symbolic import ChebyshevSymbolic, chebyshev_symbolic_basis
import sympy as sp

# Single polynomial
T5 = ChebyshevSymbolic(5)
print(T5.expression)          # 16*x**5 - 20*x**3 + 5*x

# Exact evaluation at rational point
val = T5.evaluate(sp.Rational(1, 2))   # 1/2 (exact: cos(5*pi/3))

# Derivative and integral via SymPy
dT5 = sp.diff(T5.expression, sp.Symbol('x'))
iT5 = sp.integrate(T5.expression, sp.Symbol('x'))

# Generate basis [T_0, T_1, ..., T_6]
basis = chebyshev_symbolic_basis(6)
for poly in basis:
    print(f"T_{poly.n}(x) = {poly.expression}")
```

Standalone Functions

Function	Signature	Description
‘chebyshev_symbolic_basis()‘	‘(max_n) -> List[...]‘	Basis of ChebyshevSymbolic objects
‘generate_sympy_chebyshev()‘	‘(n) -> sympy.Basic‘	Single symbolic expression
‘get_sympy_chebyshev_basis()‘	‘(max_n) -> List[sympy.Basic]‘	List of symbolic expressions

---

2.2 Layer 2 — High Precision (mpmath)

Module: `chebyshev.high_precision`

Class: **`ChebyshevMPMath(n, dps=50)`** Arbitrary-precision Chebyshev T_n(x) evaluation using mpmath with context-managed precision via `mp.workdps()`.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘dps‘	‘int‘	‘50‘	Decimal places of precision

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> mp.mpf‘	Evaluate T_n(x) at given precision
‘.derivative()‘	‘(x) -> mp.mpf‘	Numerical derivative via ‘mp.diff‘
‘.integral()‘	‘(x) -> mp.mpf‘	Definite integral from 0 to x via ‘mp.quad‘
‘.get_coefficients()‘	‘() -> List[mp.mpf]‘	Monomial coefficients [c_0, ..., c_n]

```
from chebyshev.high_precision import ChebyshevMPMath, get_mpmath_chebyshev_basis

# Single polynomial with 100 decimal places
T5_hp = ChebyshevMPMath(5, dps=100)
val   = T5_hp.evaluate(0.5)           # mpf at 100 dps
deriv = T5_hp.derivative(0.5)         # T_5'(0.5) high precision
integ = T5_hp.integral(0.75)          # integral from 0 to 0.75

# Coefficients as mpmath floats
coeffs = T5_hp.get_coefficients()     # [mpf('0'), mpf('5'), mpf('-20'), ...]

# Generate basis with custom precision
basis = get_mpmath_chebyshev_basis(6, dps=80)
```

2.3 Layer 3 — Numerical (NumPy)

Module: `chebyshev.numerical`

Class: **`ChebyshevPolynomial(n)`** Fast double-precision evaluation using three-term recurrence with caching.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Methods:

Method	Signature	Description
‘.__call__()‘	‘(x: float) -> float‘	Evaluate T_n(x)
‘.derivative()‘	‘(x: float) -> float‘	Derivative via stable series evaluation
‘.integral()‘	‘(x: float) -> float‘	Antiderivative via stable series eval

Properties:

Property	Return Type	Description

‘.coefficients‘	‘np.ndarray‘	Monomial coefficients [c_0,...,c_n]
‘.n‘	‘int‘	Degree of polynomial

```python
from chebyshev.numerical import ChebyshevPolynomial, chebyshev_coefficients

# Single polynomial
T5 = ChebyshevPolynomial(5)
print(T5(0.5))                    # 0.5
print(T5.derivative(0.5))         # T_5'(0.5)
print(T5.coefficients)            # [0., 5., -20., 0., 16.]

# Standalone functions
coeffs    = chebyshev_coefficients(5)           # [0.0, 5.0, -20.0, 0.0, 16.0]
deriv_val = chebyshev_derivative_stable(5, 0.5)
integ_val = chebyshev_integral_stable(5, 0.5)

# NumPy array output
arr   = generate_numpy_chebyshev(5)             # np.array([0., 5., -20., 0., 16.])
basis = get_numpy_chebyshev_basis(6)            # [np.ndarray, ...]
```

Class: `ChebyshevGenerator` (Internal) Cached engine for coefficient generation used by ChebyshevPolynomial via the module-level singleton `_GEN`.

Method	Signature	Description
‘.get_derivative_series()‘	‘(n) -> List[float]‘	Derivative series coefficients
‘.get_integral_series()‘	‘(n) -> List[float]‘	Integral series coefficients
‘.evaluate_series()‘	‘(x, series) -> float‘	Clenshaw-style series evaluation
‘.get_monomial_coefficients()‘	‘(n) -> List[float]‘	Monomial coefficients with cache

---

2.4 Layer 4 — Integration / Quadrature (NumPy)

Module: `chebyshev.integration`

Class: `ChebyshevQuadrature`

Method	Signature	Description
‘.get_extrema_points()‘	‘(n) -> List[float]‘	Chebyshev extrema cos(k*pi/n)
‘.clenshaw_curtis_quadrature()‘	‘(f, n) -> float‘	Clenshaw-Curtis quadrature on [-1, 1]
‘.integrate_on()‘	‘(f, a, b, n=32) -> float‘	Integrate f over arbitrary [a, b]

Standalone Functions

Function	Signature	Description
‘clencurt()‘	‘(n) -> (nodes, weights)‘	Clenshaw-Curtis nodes and weights
‘clencurt_quadrature()‘	‘(f, n) -> float‘	Integrate f on [-1, 1]

‘clencurt_integrate_interval()“(f, a, b, n=32) -> float‘		Integrate f over [a, b] with Jacobian
‘map_nodes_to_interval()‘	‘(nodes, a, b) -> np.ndarray‘	Linear map from [-1, 1] to [a, b]

```python
from chebyshev.integration import ChebyshevQuadrature, clencurt_quadrature
import numpy as np

# Clenshaw-Curtis quadrature on [-1, 1]
result = clencurt_quadrature(lambda x: np.exp(x), n=64)

# Arbitrary interval integration
quad    = ChebyshevQuadrature()
result  = quad.integrate_on(lambda x: x**3 + 2*x, a=-2.0, b=3.0, n=32)

# Extrema points (Chebyshev nodes for interpolation)
extrema = quad.get_extrema_points(10)   # [cos(k*pi/10) for k in range(11)]
```

2.5 Layer 4 — High-Precision Integration (mpmath)

Module: `chebyshev.integration_mp`

Provides arbitrary-precision equivalents of all routines in `integration.py`, with user-controlled decimal precision via the `dps` parameter. Results are cached at module level keyed by `(n, type, dps)`.

Class: `GaussChebyshevQuadrature(n, quad_type="I", dps=80)`

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of quadrature points
‘quad_type‘	‘str‘	‘”I”‘	‘”I”‘ weight 1/sqrt(1-x^2); ‘”II”‘ weight sqrt(1-x^2)
‘dps‘	‘int‘	‘80‘	Decimal places of precision

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f: Callable[[mp.mpf], mp.mpf]) -> mp.mpf‘	Compute weighted integral

```python
from chebyshev.integration_mp import GaussChebyshevQuadrature
import mpmath as mp

# Type I: weight 1/sqrt(1-x^2), integral of f=1 gives pi
quad_I = GaussChebyshevQuadrature(32, quad_type="I", dps=80)
result = quad_I.integrate(lambda x: mp.mpf(1))   # ~pi

# Type II: weight sqrt(1-x^2), integral of f=1 gives pi/2
quad_II = GaussChebyshevQuadrature(32, quad_type="II", dps=80)
result = quad_II.integrate(lambda x: mp.mpf(1))  # ~pi/2
```

Class: `ClenshawCurtisMP(n, dps=80)` Clenshaw-Curtis quadrature with mpmath-precision weights using direct cosine series for stability.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of intervals
‘dps‘	‘int‘	‘80‘	Decimal places

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f) -> mp.mpf‘	Integrate f on [-1, 1]
‘.integrate_on_interval()‘	‘(f, a, b) -> mp.mpf‘	Integrate f over [a, b] with Jacobian

```
from chebyshev.integration_mp import ClenshawCurtisMP
import mpmath as mp

cc = ClenshawCurtisMP(64, dps=80)
result = cc.integrate(lambda x: mp.exp(x))                      # [-1, 1]
result = cc.integrate_on_interval(lambda x: x**2, 0, 10, )      # [0, 10]
```

Class: `ChebyshevProjectionMP(max_degree, dps=80)` Project a function onto the Chebyshev basis using Discrete Cosine Transform (DCT-I) logic.

Parameter	Type	Default	Description
‘max_degree‘	‘int‘	—	Maximum polynomial degree
‘dps‘	‘int‘	‘80‘	Decimal places

Methods:

Method	Signature	Description
‘.project()‘	‘(f) -> List[mp.mpf]‘	Compute expansion coefficients a_k
‘.approximate()‘	‘(x, coeffs) -> mp.mpf‘	Evaluate series at point x (Clenshaw sum)

```
from chebyshev.integration_mp import ChebyshevProjectionMP
import mpmath as mp

proj   = ChebyshevProjectionMP(16, dps=80)
f_exp  = lambda x: mp.exp(x)
coeffs = proj.project(f_exp)

# Reconstruct at test points
for x in [-0.8, 0.0, 0.5]:
    approx = float(proj.approximate(x, coeffs))
```

Standalone Functions (integration_mp)

Function	Signature	Description
‘clencurt_mp()‘	‘(f, n, dps=80) -> mp.mpf‘	Clenshaw-Curtis on [-1, 1]
‘clencurt_mp_interval()‘	‘(f, a, b, n, dps=80) -> mp.mpf‘	Clenshaw-Curtis on [a, b]
‘gauss_chebyshev_mp()‘	‘(f, n, quad_type=”I”, dps=80)‘	Gauss-Chebyshev quadrature
‘chebyshev_transform_mp()‘	‘(f, max_degree, dps=80)‘	Chebyshev projection coefficients

‘inverse_chebyshev_transform()‘	‘(coeffs, x, dps=80) -> mp.mpf‘	Reconstruct from coefficients
‘get_nodes_weights_mp()‘	‘(n, dps=80) -> (nodes, weights)‘	CC nodes and weights
‘map_nodes_to_interval_mp()‘	‘(nodes, a, b, dps=80)‘	Map nodes to [a, b]
‘clencurt_quadrature_float()‘	‘(f, n, dps=80) -> float‘	Float64 bridge: high-prec internal calc
‘chebyshev_transform_float()‘	‘(f, max_degree, dps=80) -> ndarray‘	Float64 bridge: returns np.ndarray

---

3. Hermite Polynomials H_n(x)

Domain: (-inf, +inf)
Weight function: w(x) = exp(-x^2)
Recurrence: H_{n+1}(x) = 2x H_n(x) - 2n H_{n-1}(x), with H_0 = 1, H_1 = 2x
Convention: Physicists' convention (not probabilists')

3.1 Layer 1 — Symbolic (SymPy)

Module: `hermite.symbolic`

Class: `HermiteSymbolic(n)` Exact Hermite polynomial H_n(x) using SymPy's `sp.hermite()` (physicists' convention).

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Properties:

Property	Return Type	Description
‘.expression‘	‘sympy.Basic‘	Symbolic expression for H_n(x)
‘.n‘	‘int‘	Degree of polynomial

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> sympy.Basic‘	Substitute x into expression

```
from hermite.symbolic import HermiteSymbolic, hermite_symbolic_basis
import sympy as sp

H4 = HermiteSymbolic(4)
print(H4.expression)          # 16*x**4 - 48*x**2 + 12

# Exact evaluation
val = H4.evaluate(sp.Rational(1, 2))   # exact rational result

# Derivative via SymPy
dH4 = sp.diff(H4.expression, sp.Symbol('x'))

# Generate basis [H_0, ..., H_6]
basis = hermite_symbolic_basis(6)
```

3.2 Layer 2 — High Precision (mpmath)

Module: `hermite.high_precision`

Class: `HermiteMPMath(n, dps=50)` Arbitrary-precision Hermite H_n(x) evaluation using mpmath.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘dps‘	‘int‘	‘50‘	Decimal places

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> mp.mpf‘	Evaluate H_n(x) at precision
‘.__call__()‘	‘(x) -> mp.mpf‘	Alias for .evaluate()

```
from hermite.high_precision import HermiteMPMath, hermite_high_precision_basis

H4_hp = HermiteMPMath(4, dps=100)
val   = H4_hp.evaluate(0.5)           # high-precision result
print(H4_hp(1.0))                     # callable interface

# Basis with custom precision
basis = hermite_high_precision_basis(6, dps=80)
```

3.3 Layer 3 — Numerical (NumPy)

Module: `hermite.numerical`

Class: `HermitePolynomial(n)` Fast double-precision evaluation using three-term recurrence with coefficient caching.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> float \{}	np.ndarray‘
‘.__call__()‘	‘(x) -> ...‘	Alias for .evaluate()

Static Method:

Method	Signature	Description
‘.evaluate()‘ (static)	‘(x, n) -> float \{}	np.ndarray‘

Properties:

Property	Return Type	Description
‘.coefficients_ascending‘	‘List[float]‘	Monomial coefficients [c_0,...,c_n]
‘.n‘	‘int‘	Degree of polynomial

```
from hermite.numerical import HermitePolynomial, hermite_numerical_basis
import numpy as np
```

```
# Single polynomial
H4 = HermitePolynomial(4)
print(H4(0.5))                          # evaluate at scalar
print(H4(np.array([0.0, 0.5, 1.0])))    # vectorized evaluation

# Static method (no instantiation needed)
val = HermitePolynomial.evaluate(0.5, 4)

# Coefficients
coeffs = H4.coefficients_ascending      # [12.0, -48.0, 0.0, 0.0, 16.0]

# Basis
basis = hermite_numerical_basis(6)      # [H_0, ..., H_6]
```

3.4 Layer 4 — Integration / Quadrature

Module: `hermite.integration`

Class: `GaussHermiteQuadrature(n, use_mpmath=False, dps=80)` Gauss-Hermite quadrature for integrals of the form integral_{-inf}^{+inf} f(x) exp(-x^2) dx. Uses Golub-Welsch eigendecomposition of the Jacobi matrix.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of quadrature points
‘use_mpmath‘	‘bool‘	‘False‘	Use mpmath for node/weight computation
‘dps‘	‘int‘	‘80‘	Precision when using mpmath

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f: Callable) -> float‘	Compute integral with weight exp(-x^2)

```
from hermite.integration import GaussHermiteQuadrature
import numpy as np

# Standard double-precision quadrature (n=20 points)
gh = GaussHermiteQuadrature(20)
result = gh.integrate(lambda x: 1.0)   # should be sqrt(pi)

# High-precision node/weight computation
gh_hp = GaussHermiteQuadrature(50, use_mpmath=True, dps=100)
```

Class: `HermiteProjection(max_degree, use_mpmath=False)` Project a function onto the Hermite basis using corrected physicists' normalization.

Norm squared: ||H_k||^2 = sqrt(pi) * 2^k * k!

Parameter	Type	Default	Description
‘max_degree‘	‘int‘	—	Maximum polynomial degree
‘use_mpmath‘	‘bool‘	‘False‘	Use mpmath for quadrature

Methods:

Method	Signature	Description
‘.project()‘	‘(f) -> np.ndarray‘	Compute expansion coefficients
‘.approximate()‘	‘(x, coeffs) -> np.ndarray‘	Reconstruct function from coefficients

```
from hermite.integration import HermiteProjection
import numpy as np

proj = HermiteProjection(max_degree=10)
f    = lambda x: np.exp(-x**2 / 4)
coeffs = proj.project(f)                    # expansion coefficients
approx = proj.approximate(np.linspace(-5, 5, 100), coeffs)   # reconstruction
```

Standalone Functions

Function	Signature	Description
‘hermite_transform()‘	‘(f, max_degree, use_mpmath)‘	Hermite expansion coefficients
‘inverse_hermite_transform()‘	‘(coeffs, x) -> np.ndarray‘	Reconstruct from coefficients

```
from hermite.integration import hermite_transform, inverse_hermite_transform
import numpy as np

coeffs = hermite_transform(lambda x: np.exp(-x**2/4), max_degree=10)
reconstructed = inverse_hermite_transform(coeffs, np.linspace(-5, 5, 100))
```

---

4. Laguerre Polynomials L_n^(alpha)(x)

Domain: [0, +inf)
Weight function: w(x) = x^alpha * exp(-x)
Recurrence (generalized): (k+1)L_{k+1}^{(a)} = [(2k+a+1-x)L_k^{(a)} - (k+a)L_{k-1}^{(a)}]
Special case: alpha=0 gives standard Laguerre L_n(x)

4.1 Layer 1 — Symbolic (SymPy)

Module: `laguerre.symbolic`

Class: **`LaguerreSymbolic(n, alpha=0.0)`** Exact generalized Laguerre polynomial L_n^(alpha)(x) using SymPy. Uses `sp.laguerre()` for alpha=0 and `sp.assoc_laguerre()` otherwise.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘alpha‘	‘float‘	‘0.0‘	Parameter alpha > -1

Properties:

Property	Return Type	Description

‘.expression‘	‘sympy.Basic‘	Symbolic expression for L_n^(a)(x)
‘.n‘	‘int‘	Degree of polynomial
‘.alpha‘	‘float‘	The alpha parameter

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> sympy.Basic‘	Substitute x into expression

```
from laguerre.symbolic import LaguerreSymbolic, laguerre_symbolic_basis
import sympy as sp

# Standard Laguerre (alpha=0)
L3 = LaguerreSymbolic(3)
print(L3.expression)          # uses sp.laguerre(n, x)

# Generalized Laguerre (alpha != 0)
L3a = LaguerreSymbolic(3, alpha=1.5)
print(L3a.expression)         # uses sp.assoc_laguerre(n, alpha, x)

# Exact evaluation
val = L3.evaluate(sp.Rational(1, 2))

# Generate basis [L_0, ..., L_6] with alpha=0.5
basis = laguerre_symbolic_basis(6, alpha=0.5)
```

4.2 Layer 2 — High Precision (mpmath)

Module: `laguerre.high_precision`

Class: **`LaguerreMPMath(n, alpha=0.0, dps=50)`** Arbitrary-precision generalized Laguerre L_n^(alpha)(x) using mpmath's `mp.laguerre(n, alpha, x)`.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘alpha‘	‘float‘	‘0.0‘	Parameter alpha > -1
‘dps‘	‘int‘	‘50‘	Decimal places of precision

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> mp.mpf‘	Evaluate L_n^(a)(x)

```
from laguerre.high_precision import LaguerreMPMath, laguerre_high_precision_basis

# Standard Laguerre with 100 dps
L3_hp = LaguerreMPMath(3, alpha=0.0, dps=100)
val   = L3_hp.evaluate(2.5)

# Generalized Laguerre
L3a_hp = LaguerreMPMath(3, alpha=1.5, dps=80)
val_a  = L3a_hp.evaluate(0.75)
```

```
# Basis with custom precision
basis = laguerre_high_precision_basis(6, alpha=0.5, dps=80)
```

4.3 Layer 3 — Numerical (NumPy)

Module: `laguerre.numerical`

Class: **`LaguerrePolynomial(n)`** Fast standard Laguerre L_n(x) evaluation (alpha = 0). Subclass of GeneralizedLaguerrePolynomial.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Class: **`GeneralizedLaguerrePolynomial(n, alpha=0.0)`** Fast generalized Laguerre L_n^(alpha)(x) evaluation using three-term recurrence.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘alpha‘	‘float‘	‘0.0‘	Parameter alpha > -1

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> float \{}	np.ndarray‘
‘.__call__()‘	‘(x) -> ...‘	Alias for .evaluate()
‘.evaluate_static()‘ (static)	‘(x, n, alpha) -> ...‘	Static evaluation

Properties:

Property	Return Type	Description
‘.coefficients_ascending‘	‘List[float]‘	Monomial coefficients via gamma formula

```
from laguerre.numerical import LaguerrePolynomial, GeneralizedLaguerrePolynomial,
    ↪laguerre_numerical_basis

# Standard Laguerre
L3 = LaguerrePolynomial(3)
print(L3(2.5))                          # evaluate at scalar

# Generalized Laguerre
L3a = GeneralizedLaguerrePolynomial(3, alpha=1.5)
print(L3a(0.75))                        # evaluate with alpha parameter

# Static method (no instantiation)
val = GeneralizedLaguerrePolynomial.evaluate_static(2.5, 3, alpha=0.0)

# Coefficients via gamma function formula
coeffs = L3.coefficients_ascending

# Basis
basis = laguerre_numerical_basis(6, alpha=0.5)   # [L_0^(a), ..., L_6^(a)]
```

4.4 Layer 4 — Integration / Quadrature

Module: `laguerre.integration`

Class: **`LaguerreQuadrature(n, alpha=0.0, use_mpmath=False)`** Gauss-Laguerre quadrature for integrals integral_0^inf f(x) x^alpha exp(-x) dx. Uses Golub-Welsch eigendecomposition of the Jacobi matrix with diagonal entries 2k+alpha+1 and off-diagonal sqrt(k*(k+alpha)).

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of quadrature points
‘alpha‘	‘float‘	‘0.0‘	Weight parameter
‘use_mpmath‘	‘bool‘	‘False‘	Use mpmath for node/weight computation

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f: Callable) -> float‘	Compute integral with weight x^a exp(-x)

Properties:

Property	Return Type	Description
‘.nodes‘	‘np.ndarray‘	Quadrature nodes in [0, inf)
‘.weights‘	‘np.ndarray‘	Quadrature weights

```
from laguerre.integration import LaguerreQuadrature

# Standard Gauss-Laguerre (alpha=0)
ql = LaguerreQuadrature(20, alpha=0.0)
result = ql.integrate(lambda x: 1.0 / (1.0 + x**2))   # integral from 0 to inf

print(ql.nodes)     # quadrature nodes in [0, inf)
print(ql.weights)   # corresponding weights

# Generalized with mpmath precision
ql_hp = LaguerreQuadrature(30, alpha=1.5, use_mpmath=True)
```

Class: **`LaguerreBasis(max_n) / GeneralizedLaguerreBasis(max_n, alpha=0.0)`** Complete basis with built-in quadrature and norm computation.

Method	Signature	Description
‘.norm_squared()‘	‘(n) -> float‘	

Properties:

Property	Type	Description
‘.quad‘	‘LaguerreQuadrature‘	Built-in quadrature object
‘.polys‘	‘List[GeneralizedLaguerrePolynomial]‘	Polynomial instances

```
from laguerre.integration import LaguerreBasis, GeneralizedLaguerreBasis

basis     = LaguerreBasis(10)                    # standard, alpha=0
gen_basis = GeneralizedLaguerreBasis(10, alpha=1.5)

norm_sq = gen_basis.norm_squared(5)              # ||L_5^(1.5)||^2
```

Standalone Functions

Function	Signature	Description
‘compute_roots()‘	‘(n, alpha=0.0)‘	Quadrature nodes (roots of L_n^(a))
‘gauss_quadrature_weights()‘	‘(n, alpha=0.0)‘	Quadrature weights
‘function_projection()‘	‘(f, max_n, alpha=0.0) -> np.ndarray‘	Expansion coefficients
‘function_approximation()‘	‘(f, max_n, alpha=0.0) -> Callable‘	Reconstructed approximation function

```python
from laguerre.integration import function_projection, function_approximation
import numpy as np

# Project f(x) = exp(-x/3) onto Laguerre basis up to degree 15
coeffs = function_projection(lambda x: np.exp(-x/3), max_n=15)

# Get callable approximation
approx = function_approximation(lambda x: np.exp(-x/3), max_n=15)
print(approx(2.0))   # evaluate approximation at x=2
```

---

5. Legendre Polynomials P_n(x)

Domain: [-1, 1]
Weight function: w(x) = 1 (uniform)
Recurrence: (n+1)P_{n+1}(x) = (2n+1)x P_n(x) - n P_{n-1}(x), with P_0=1, P_1=x

5.1 Layer 1 — Symbolic (SymPy)

Module: `legendre.symbolic`

Class: **`LegendreSymbolic(n)`** Exact Legendre polynomial P_n(x) using SymPy's `sp.legendre()`. Provides exact rational coefficients, symbolic differentiation, and integration.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Properties:

Property	Return Type	Description
‘.expression‘	‘sympy.Basic‘	Symbolic expression for P_n(x)
‘.coefficients_ascending‘	‘List[sympy.Rational]‘	Exact rational coefficients [c_0,...,c_n]
‘.coefficients_descending‘	‘List[sympy.Rational]‘	Coefficients from highest to lowest power

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> sympy.Basic‘	Substitute x into expression
‘.derivative()‘	‘() -> sympy.Basic‘	Symbolic derivative P_n’(x)
‘.integral()‘	‘() -> sympy.Basic‘	Symbolic antiderivative integral P_n(x)dx

```python
from legendre.symbolic import LegendreSymbolic, legendre_symbolic_basis
import sympy as sp

P4 = LegendreSymbolic(4)
print(P4.expression)                    # (35*x**4 - 30*x**2 + 3)/8

# Exact rational coefficients
asc  = P4.coefficients_ascending        # [3/8, 0, -15/4, 0, 35/8]
desc = P4.coefficients_descending       # [35/8, 0, -15/4, 0, 3/8]

# Symbolic derivative and integral
dP4 = P4.derivative()                   # (35*x**3 - 15*x)/2
iP4 = P4.integral()                     # antiderivative with constant=0

# Exact evaluation
val = P4.evaluate(sp.Rational(1, 3))    # exact rational result

# Basis [P_0, ..., P_6]
basis = legendre_symbolic_basis(6)
```

Standalone Functions

Function	Signature	Description
‘legendre_symbolic_basis()‘	‘(max_n) -> List[...]‘	Basis of LegendreSymbolic instances
‘generate_sympy_legendre()‘	‘(n) -> sympy.Basic‘	Single symbolic expression
‘get_sympy_legendre_basis()‘	‘(max_n) -> List[sympy.Basic]‘	List of symbolic expressions

---

5.2 Layer 2 — High Precision (mpmath)

Module: `legendre.high_precision`

Class: `LegendreMPMath(n, dps=50)` Arbitrary-precision Legendre P_n(x) using mpmath with coefficient extraction via three-term recurrence at full precision.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Non-negative degree
‘dps‘	‘int‘	‘50‘	Decimal places

Methods:

Method	Signature	Description
‘.evaluate()‘	‘(x) -> mp.mpf‘	Evaluate P_n(x) at precision
‘.derivative_value()‘	‘(x) -> mp.mpf‘	Numerical derivative via mp.diff

Properties:

Property	Return Type	Description
‘.coefficients_ascending‘	‘List[mp.mpf]‘	High-precision monomial coefficients

```python
from legendre.high_precision import LegendreMPMath, legendre_high_precision_basis

P10_hp = LegendreMPMath(10, dps=100)
val   = P10_hp.evaluate(0.5)                # 100 decimal places
deriv = P10_hp.derivative_value(0.75)       # high-precision derivative

# Coefficients at high precision
coeffs = P10_hp.coefficients_ascending      # List[mp.mpf]

# Basis with custom precision
basis = legendre_high_precision_basis(6, dps=80)
```

Standalone Functions

Function	Signature	Description
‘generate_mpmath_legendre()‘	‘(n, dps=50) -> LegendreMPMath‘	Single high-precision object
‘get_mpmath_legendre_basis()‘	‘(max_n, dps=50)‘	Basis of objects
‘evaluate_mpmath_legendre()‘	‘(n, x, dps=50) -> mp.mpf‘	Direct evaluation
‘get_mpmath_legendre_coefficients‘	‘(n, dps=50) -> List[mp.mpf]‘	Ascending coefficients
‘get_mpmath_legendre_coefficients_ascending()‘	‘(n, dps=50)‘	Same as above
‘get_mpmath_legendre_coefficients_descending()‘	‘(n, dps=50)‘	Descending coefficients

---

5.3 Layer 3 — Numerical (NumPy)

Module: `legendre.numerical`

Class: `LegendrePolynomial(n)` Fast double-precision evaluation using three-term recurrence with coefficient caching via LegendreGenerator singleton.

Parameter	Type	Description
‘n‘	‘int‘	Non-negative degree

Methods:

Method	Signature	Description
‘.evaluate()‘ (static)	‘(x, n) -> float \{}	np.ndarray‘
‘.__call__()‘	‘(x) -> ...‘	Alias for evaluate
‘.derivative_coefficients()‘	‘() -> List[float]‘	Coefficients of P_n’(x) descending
‘.integral_coefficients()‘	‘() -> List[float]‘	Coefficients of integral P_n(x)dx desc.

Properties:

Property	Return Type	Description
‘.coefficients_descending‘	‘List[float]‘	[c_n, c_{n-1}, ..., c_0]
‘.coefficients_ascending‘	‘List[float]‘	[c_0, c_1, ..., c_n]

```python
from legendre.numerical import LegendrePolynomial, get_numpy_legendre_basis
import numpy as np

# Single polynomial
```

```
P4 = LegendrePolynomial(4)
print(P4(0.5))                              # evaluate at scalar
print(P4(np.array([0.0, 0.5, 1.0])))        # vectorized evaluation

# Static method (no instantiation needed)
val = LegendrePolynomial.evaluate(0.5, 4)

# Coefficients in both orders
desc = P4.coefficients_descending           # [35/8, 0, -30/8, 0, 3/8] as floats
asc  = P4.coefficients_ascending            # [3/8, 0, -30/8, 0, 35/8]

# Derivative and integral coefficients
deriv_coeffs = P4.derivative_coefficients() # descending order
integ_coeffs = P4.integral_coefficients()   # descending order

# Basis
basis = get_numpy_legendre_basis(6)         # [LegendrePolynomial instances]
```

Class: `LegendreGenerator` (Internal) Cached generator for coefficient computation, accessed via singleton `_LEGENDRE_GEN`.

Method	Signature	Description
‘.get_coefficients_descending()‘	‘(n) -> List[float]‘	Descending monomial coefficients
‘.get_coefficients_ascending()‘	‘(n) -> List[float]‘	Ascending monomial coefficients

Standalone Functions (numerical)

Function	Signature	Description
‘legendre_polynomial()‘	‘(n, x) -> float‘	Evaluate P_n(x)
‘evaluate_numpy_legendre()‘	‘(n, x) -> float \{}	np.ndarray‘
‘legendre_coefficients()‘	‘(n) -> List[float]‘	Descending coefficients (alias)
‘legendre_coefficients_descending()‘	‘(n) -> List[float]‘	Descending monomial coeffs
‘legendre_coefficients_ascending()‘	‘(n) -> List[float]‘	Ascending monomial coeffs
‘legendre_derivative()‘	‘(n) -> List[float]‘	Derivative coefficients desc.
‘legendre_integral()‘	‘(n) -> List[float]‘	Integral coefficients desc.
‘generate_numpy_legendre()‘	‘(n) -> np.ndarray‘	Descending coeffs as array
‘get_numpy_legendre_basis()‘	‘(max_n)‘	Basis of LegendrePolynomial objs

---

5.4 Layer 4 — Integration / Quadrature

Module: `legendre.integration`

Class: `GaussLegendreQuadrature` (static methods) Standard double-precision Gauss-Legendre quadrature computation with two algorithms.

Static Method	Signature	Description

‘.golub_welsch()‘	‘(n) -> (nodes, weights)‘	Golub-Welsch eigendecomposition method
‘.newton_raphson()‘	‘(n) -> (nodes, weights)‘	Newton-Raphson root-finding method

```
from legendre.integration import GaussLegendreQuadrature

# Golub-Welsch method (eigendecomposition of Jacobi matrix)
nodes, weights = GaussLegendreQuadrature.golub_welsch(20)

# Newton-Raphson method (Trefethen initial guesses)
nodes_nr, weights_nr = GaussLegendreQuadrature.newton_raphson(20)
```

Class: `LegendreQuadrature(n, use_mpmath=False, dps=80)` Gauss-Legendre quadrature with automatic precision selection. Nodes and weights are returned as float64 NumPy arrays (even when computed via mpmath for stability). For integrals of the form integral_{-1}^{1} f(x) dx.

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of quadrature points
‘use_mpmath‘	‘bool‘	‘False‘	Use mpmath for stable node computation
‘dps‘	‘int‘	‘80‘	Precision when using mpmath

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f: Callable) -> float‘	Compute integral_{-1}^{1} f(x) dx

Properties:

Property	Return Type	Description
‘.nodes‘	‘np.ndarray‘	Quadrature nodes in (-1, 1)
‘.weights‘	‘np.ndarray‘	Quadrature weights (sum to 2.0)

```
from legendre.integration import LegendreQuadrature
import numpy as np

# Standard double-precision (n=20 points)
gl = LegendreQuadrature(20)
result = gl.integrate(lambda x: np.exp(x))    # integral from -1 to 1

print(gl.nodes)     # nodes in (-1, 1)
print(gl.weights)   # weights summing to 2.0

# High-precision node computation (still returns float64 arrays)
gl_hp = LegendreQuadrature(80, use_mpmath=True, dps=100)
```

Class: `HighPrecisionGaussLegendre(n, dps=80)` True arbitrary-precision Gauss-Legendre quadrature. Nodes and weights remain as mpmath.mpf objects for full precision throughout the computation. Accepts either mpmath-compatible callables or SymPy expressions (auto-lambdified).

Parameter	Type	Default	Description
‘n‘	‘int‘	—	Number of quadrature points
‘dps‘	‘int‘	‘80‘	Decimal places

Methods:

Method	Signature	Description
‘.integrate()‘	‘(f_or_expr) -> mp.mpf‘	Integrate callable or SymPy expression

Properties:

Property	Return Type	Description
‘.nodes‘	‘List[mp.mpf]‘	High-precision nodes
‘.weights‘	‘List[mp.mpf]‘	High-precision weights

```
from legendre.integration import HighPrecisionGaussLegendre
import mpmath as mp
import sympy as sp

# True arbitrary-precision quadrature
hpgl = HighPrecisionGaussLegendre(50, dps=100)

# Integrate with mpmath-compatible function
result = hpgl.integrate(lambda x: mp.exp(x))

# Integrate a SymPy expression (auto-lambdified to mpmath)
x_sym = sp.Symbol('x')
result2 = hpgl.integrate(sp.sin(x_sym)**2 + sp.exp(-x_sym**2))

print(hpgl.nodes[0])    # mpf object with full precision
```

Standalone Functions (integration)

Function	Signature	Description
‘gauss_legendre()‘	‘(n) -> (nodes, weights)‘	Double-precision Golub-Welsch
‘gauss_legendre_newton()‘	‘(n) -> (nodes, weights)‘	Newton-Raphson method
‘gauss_legendre_golub_welsch()‘	‘(n) -> (nodes, weights)‘	Explicit Golub-Welsch alias
‘gauss_legendre_high_precision()‘	‘(n, dps=80) -> HighPrecisionGaussLegendre‘	Factory for HP quadrature

```
from legendre.integration import gauss_legendre, gauss_legendre_newton

# Quick access to nodes and weights
nodes, weights = gauss_legendre(20)           # Golub-Welsch
nodes_nr, w_nr  = gauss_legendre_newton(20)   # Newton-Raphson

# High-precision factory
hpgl = gauss_legendre_high_precision(50, dps=100)
```

---

6. Cross-Family Comparison

6.1 Domain and Weight Function Summary

Family	Domain	Weight w(x)	Orthogonality Integral
Chebyshev	[-1, 1]	(1-x^2)^(-1/2)	integral_{-1}^{1} T_n T_m w(x) dx
Hermite	(-inf,+inf)	exp(-x^2)	integral H_n H_m exp(-x^2) dx
Laguerre	[0, +inf)	x^alpha * exp(-x)	integral L_n^(a) L_m^(a) x^a exp(-x) dx
Legendre	[-1, 1]	1	integral_{-1}^{1} P_n P_m dx

6.2 Import Pattern Comparison

```
# CHEBYSHEV ------------------------------------------------------------
from chebyshev.symbolic import ChebyshevSymbolic, chebyshev_symbolic_basis
from chebyshev.high_precision import ChebyshevMPMath, get_mpmath_chebyshev_basis
from chebyshev.numerical import ChebyshevPolynomial, generate_numpy_chebyshev
from chebyshev.integration import ChebyshevQuadrature, clencurt_quadrature

# HERMITE --------------------------------------------------------------
from hermite.symbolic import HermiteSymbolic, hermite_symbolic_basis
from hermite.high_precision import HermiteMPMath, hermite_high_precision_basis
from hermite.numerical import HermitePolynomial, hermite_numerical_basis
from hermite.integration import GaussHermiteQuadrature, HermiteProjection

# LAGUERRE -------------------------------------------------------------
from laguerre.symbolic import LaguerreSymbolic, laguerre_symbolic_basis
from laguerre.high_precision import LaguerreMPMath, laguerre_high_precision_basis
from laguerre.numerical import LaguerrePolynomial, GeneralizedLaguerrePolynomial
from laguerre.integration import LaguerreQuadrature, function_projection

# LEGENDRE -------------------------------------------------------------
from legendre.symbolic import LegendreSymbolic, legendre_symbolic_basis
from legendre.high_precision import LegendreMPMath, legendre_high_precision_basis
from legendre.numerical import LegendrePolynomial, get_numpy_legendre_basis
from legendre.integration import LegendreQuadrature, HighPrecisionGaussLegendre
```

6.3 Unified Evaluation Pattern

All four families support the same three-layer evaluation pattern:

```
import numpy as np

# LAYER 1: Symbolic (exact) ----------------------------------------
from chebyshev.symbolic import ChebyshevSymbolic
T5 = ChebyshevSymbolic(5)
print(T5.expression)              # exact symbolic form

# LAYER 2: High Precision (arbitrary dps) --------------------------
from chebyshev.high_precision import ChebyshevMPMath
T5_hp = ChebyshevMPMath(5, dps=100)
print(T5_hp.evaluate(0.5))        # 100 decimal places

# LAYER 3: Numerical (fast, double precision) ----------------------
from chebyshev.numerical import ChebyshevPolynomial
T5_np = ChebyshevPolynomial(5)
print(T5_np(0.5))                 # fast float64 evaluation
```

```python
# LAYER 4: Quadrature (integration) --------------------------------
from chebyshev.integration import clencurt_quadrature
result = clencurt_quadrature(lambda x: np.exp(x), n=64)
```

---

7. Choosing the Right Layer

Decision Guide

Use Case	Recommended Layer	Reason
Deriving formulas, proofs, exact coefficients	**Layer 1** (SymPy)	Exact rational arithmetic
Verifying numerical results	**Layer 1** (SymPy)	Ground-truth reference
High-precision computation (>53 bits)	**Layer 2** (mpmath)	Configurable decimal precision
Sensitivity analysis, ill-conditioned problems	**Layer 2** (mpmath)	Avoids floating-point accumulation errors
Performance-critical loops	**Layer 3** (NumPy)	Fast C-backed vectorized operations
Large-scale batch evaluation	**Layer 3** (NumPy)	Array broadcasting, caching
Numerical integration	**Layer 4**	Gauss-type quadrature with auto-selection
Function approximation / spectral methods	**Layer 4**	Projection and reconstruction utilities

Precision Comparison Example

```python
import numpy as np
from legendre.symbolic import LegendreSymbolic
from legendre.high_precision import LegendreMPMath
from legendre.numerical import LegendrePolynomial
import sympy as sp
import mpmath as mp

n = 20
x_val = sp.Rational(7, 10)   # x = 0.7

# Layer 1: Exact
P20_sym = LegendreSymbolic(n)
exact = P20_sym.evaluate(x_val)
print(f"Exact (SymPy):     {exact}")

# Layer 2: High precision (100 dps)
P20_hp = LegendreMPMath(n, dps=100)
hp_val = P20_hp.evaluate(0.7)
print(f"High-precision:    {mp.nstr(hp_val, 50)}")

# Layer 3: Double precision (float64 ~16 digits)
P20_np = LegendrePolynomial(n)
np_val = P20_np(0.7)
print(f"NumPy float64:     {np_val:.16f}")
```

Appendix A: Version Information

Package	Version
Chebyshev	‘2.0.3-hpz4‘
Hermite	‘2.0.1-hpz4-fixed‘
Laguerre	‘2.0.0-hpz4‘
Legendre	‘2.0.0-hpz4‘

Appendix B: Dependencies

```
sympy    # Layer 1 — Symbolic exact arithmetic
mpmath   # Layer 2 — Arbitrary-precision floating point
numpy    # Layer 3 — Fast numerical array operations
```

Install all dependencies:

```
pip install sympy mpmath numpy
```

Appendix C: Error Conventions

All classes validate their inputs at construction time:

Condition	Exception	Message
‘n < 0‘ or not int	‘ValueError‘	”n must be a non-negative integer”
‘alpha <= -1‘	‘ValueError‘	”alpha must be > -1”
Missing sympy/mpmath	‘ImportError‘	Installation instructions provided

Appendix D: Package-Level Imports

Each family exposes its public API through the package `__init__.py`:

```
# Chebyshev — top-level import
import chebyshev
chebyshev.ChebyshevSymbolic(5)
chebyshev.ChebyshevMPMath(5, dps=80)
chebyshev.ChebyshevPolynomial(5)
chebyshev.clencurt_quadrature(lambda x: x**2, n=32)

# Hermite — top-level import
import hermite
hermite.HermiteSymbolic(4)
hermite.HermiteMPMath(4, dps=80)
hermite.HermitePolynomial(4)
hermite.GaussHermiteQuadrature(20).integrate(lambda x: 1.0)

# Laguerre — top-level import
import laguerre
laguerre.LaguerreSymbolic(3, alpha=1.5)
laguerre.LaguerreMPMath(3, alpha=1.5, dps=80)
laguerre.GeneralizedLaguerrePolynomial(3, alpha=1.5)
laguerre.function_projection(lambda x: np.exp(-x/3), max_n=15)

# Legendre — top-level import
import legendre
legendre.LegendreSymbolic(4)
legendre.LegendreMPMath(10, dps=100)
```

```
legendre.LegendrePolynomial(4)
legendre.HighPrecisionGaussLegendre(50, dps=100).integrate(lambda x: mp.exp(x))
```